

Support Vector Machine

In this file, we will go through the details of the support vector machine (SVM) and a slightly simplified Sequential Minimal Optimization (SMO) algorithm. This will not be a complete derivation of the SVM, but we will go through certain update rules so that we could implement it.

Consider the following optimization problem:

$$\min_{\xi_i, w, b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \xi_i$$

(C is a hyperparameter that corresponds to the strength of our regularization)

$$\text{s.t. } y^{(i)}(w^T x^{(i)} + b) \geq 1 - \xi_i, \quad i=1, \dots, m$$

$$\text{The } \xi_i \text{ are } \xi_i \geq 0, \quad i=1, \dots, m$$

This is the primal optimization problem behind the L_1 regularized SVM, the derivation of which is beyond the scope of our implementation. Since our problem is convex, and Slater's condition is met, we can develop the dual optimization problem, the solution to which would equal the solution to the primal problem. Moreover, the Karush-Kuhn-Tucker (KKT) conditions give us a set of conditions that the optimum would satisfy, and by applying the KKT stationary condition, we arrive at the following dual optimization problem:

$$\max_{\alpha} W(\alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m y^{(i)} y^{(j)} \alpha_i \alpha_j \langle x^{(i)}, x^{(j)} \rangle$$

$$\text{s.t. } 0 \leq \alpha_i \leq C, \quad i=1, \dots, m$$

$$\sum_{i=1}^m \alpha_i y^{(i)} = 0$$

Note that the α_i 's are Lagrange multipliers, which come from the Lagrangian of the primal problem. Furthermore, $\langle x^{(i)}, x^{(j)} \rangle$ represents the inner product between x and z ($x^T z$). Notice that we have rewritten our problem to be in terms of inner products. Moreover, from the KKT stationary condition, we arrive at the following formula for w :

$$w = \sum_{i=1}^m \alpha_i y^{(i)} x^{(i)}$$

When we make a prediction, we compute $w^T x + b$ and predict $y=1$ if this quantity is ≥ 0 . Given our formula for w , we can rewrite our hypothesis function as

$$\begin{aligned} w^T x + b &= \left(\sum_{i=1}^m \alpha_i y^{(i)} x^{(i)} \right)^T x + b \\ &= \sum_{i=1}^m \alpha_i y^{(i)} \langle x^{(i)}, x \rangle + b \end{aligned}$$

Hence, if we have found the α_i 's, to make a prediction, we have to calculate a quantity that only depends on the inner product between the training set and our new x . This allows us to introduce a trick known as the kernel trick. The kernel trick uses a function that corresponds to an inner product between two feature vectors of a much higher dimension, e.g. recall that the Gaussian kernel is given by

$$K(x, z) = \exp\left(-\frac{\|x - z\|^2}{2\sigma^2}\right) = \phi(x)^T \phi(z)$$

This is the Gaussian/RBF Kernel, which computes the inner product of $\phi(x)$ and $\phi(z)$, where $\phi(x)$ and $\phi(z)$ live in an infinite dimensional feature space, and the kernel computes the inner product without ever representing these feature vectors, meaning that we don't sacrifice any computational efficiency. This allows us to very efficiently separate data in an infinite dimensional feature space, corresponding to a nonlinear decision boundary. In our implementation, we will use this C-Kernel.

To implement the SVM, we need to introduce the SMO algorithm, which will solve our constrained optimization problem. Since we have the constraint $\sum_{i=1}^m \alpha_i y^{(i)} = 0$, we will need to optimize over two α_i 's to make any progress.

Much of the complexity of the SMO algorithm comes from a heuristic on how to choose the two Lagrange multipliers, however, for our implementation, we will just pick two randomly.

The KKT conditions can be used to check for convergence towards the optimal point. For our optimization problem, it can be seen that the KKT conditions are:

$$\alpha_i = 0 \Rightarrow y^{(i)}(w^T x^{(i)} + b) \geq 1$$

$$\alpha_i = C \Rightarrow y^{(i)}(w^T x^{(i)} + b) \leq -1$$

$$0 \leq \alpha_i \leq C \Rightarrow y^{(i)}(w^T x^{(i)} + b) = -1$$

Having found α_i and α_j to optimize (either using our random heuristic, or the full SMO heuristic), we compute the bounds for these parameters, and then solve the constrained maximization problem.

Let's find the bounds L and H such that $L \leq \alpha_j \leq H$ in order for α_j to satisfy the constraint that $0 \leq \alpha_j \leq C$:

$$\sum_{i=1}^n \alpha_i y^{(i)} = 0 \Rightarrow \alpha_i y^{(i)} + \alpha_j y^{(j)} = \zeta \quad (\text{constant})$$

$$\alpha_j = y^{(i)} \zeta - \alpha_i y^{(i)} y^{(j)}$$

We know that $0 \leq \alpha_i \leq C$

$$\text{if } y^{(i)} \neq y^{(j)}: \quad 0 \leq y^{(i)} \zeta + \alpha_j \leq C \Rightarrow -y^{(i)} \zeta \leq \alpha_j \leq C - y^{(i)} \zeta$$

$$\alpha_j - \alpha_i \leq \alpha_j \leq C + \alpha_j - \alpha_i, \text{ and } 0 \leq \alpha_j \leq C \text{ as originally stated}$$

$$\text{if } y^{(i)} = y^{(j)}: \quad 0 \leq y^{(i)} \zeta - \alpha_j \leq C \Rightarrow -C + y^{(i)} \zeta \leq \alpha_j \leq y^{(i)} \zeta$$

$$\alpha_i + \alpha_j - C \leq \alpha_j \leq \alpha_i + \alpha_j, \text{ and } 0 \leq \alpha_j \leq C \text{ as originally stated}$$

Therefore:

$$\text{if } y^{(i)} \neq y^{(j)}: \quad L = \max(0, \alpha_i - \alpha_j), \quad H = \min(C, C + \alpha_j - \alpha_i)$$

$$\text{if } y^{(i)} = y^{(j)}: \quad L = \max(0, \alpha_i + \alpha_j - C), \quad H = \min(C, \alpha_i + \alpha_j)$$

Now that we have computed bounds for α_j , let's derive an update rule for this parameter. We will do this by maximizing $W(\alpha)$ with respect to α_j , and if α_j lives outside the bounds, we can very easily clip the value to live between L and H .

Define:

$$f(x) = \sum_{i=1}^m \alpha_i y^{(i)} K_{i,x} + b$$

$$E_i = f(x^{(i)}) - y^{(i)}$$

$$\eta = \sum K_{ii} - K_{ii} - K_{jj}$$

$$\begin{aligned} W(a) &= \sum_{l=1}^m d_l - \frac{1}{2} \sum_{l=1}^m \sum_{k \neq l} \alpha_l \alpha_k y^{(l)} y^{(k)} K_{l,k} \\ &= \alpha_i + \alpha_i - \frac{1}{2} y^{(i)^2} \alpha_i^2 K_{ii} - \frac{1}{2} y^{(i)^2} \alpha_j^2 K_{jj} \\ &\quad - \alpha_i \alpha_j y^{(i)} y^{(j)} K_{ij} - \sum_{k \neq i,j} \alpha_i \alpha_k y^{(i)} y^{(k)} K_{i,k} \\ &\quad - \sum_{k \neq i,j} \alpha_j \alpha_k y^{(j)} y^{(k)} K_{j,k} - \frac{1}{2} \sum_{l \neq i,j} \sum_{k \neq i,j} \alpha_l \alpha_k y^{(l)} y^{(k)} K_{l,k} \end{aligned}$$

(let $K_{ij} = K(x^{(i)}, x^{(j)})$, where K_{ij} is an index into the kernel matrix, created from inner products in the training set $(x^{(1)}, \dots, x^{(m)})$)

Let's represent d_i in terms of α_j :

$$\alpha_i^{old} y^{(i)} + \alpha_j^{new} y^{(j)} = \zeta = \alpha_i^{new} y^{(i)} + \alpha_j^{new} y^{(j)}$$

$$\alpha_i^{new} = \alpha_i^{old} + y^{(i)} y^{(j)} (\alpha_j^{old} - \alpha_j) \quad \text{where } \alpha_i^{new} = \alpha_j, \alpha_j^{new} = \alpha_i$$

$$\begin{aligned} W(a) &= (\alpha_i^{old} + y^{(i)} y^{(j)} (\alpha_j^{old} - \alpha_j)) + \alpha_j - \frac{1}{2} (\alpha_i^{old} + y^{(i)} y^{(j)} (\alpha_j^{old} - \alpha_j))^2 K_{ii} - \frac{1}{2} \alpha_j^2 K_{jj} \\ &\quad - (\alpha_i^{old} + y^{(i)} y^{(j)} (\alpha_j^{old} - \alpha_j)) \alpha_j y^{(i)} y^{(j)} K_{ij} - \sum_{k \neq i,j} (\alpha_i^{old} + y^{(i)} y^{(j)} (\alpha_j^{old} - \alpha_j)) \alpha_k y^{(i)} y^{(k)} K_{i,k} \\ &\quad - \sum_{k \neq i,j} \alpha_j \alpha_k y^{(j)} y^{(k)} K_{j,k} + \text{constants} \end{aligned}$$

$$\frac{\partial W(a)}{\partial \alpha_j} = -y^{(i)} y^{(j)} + 1 - (\alpha_i^{old} + y^{(i)} y^{(j)} (\alpha_j^{old} - \alpha_j)) (-y^{(i)} y^{(j)}) K_{ii} - \alpha_j K_{jj}$$

$$- \alpha_i^{old} y^{(i)} y^{(j)} K_{ij} - \alpha_j^{old} K_{ij} + 2 \alpha_j K_{ij} + \sum_{k \neq i,j} y^{(j)} y^{(k)} \alpha_k K_{j,k} - \sum_{k \neq i,j} \alpha_k y^{(i)} y^{(k)} K_{i,k}$$

$$= -y^{(i)} y^{(j)} + 1 + \alpha_i^{old} y^{(i)} y^{(j)} K_{ii} + \alpha_j^{old} K_{ii} - \alpha_j K_{ii} - \alpha_j K_{jj} - \alpha_i^{old} y^{(i)} y^{(j)} K_{ij}$$

$$- \alpha_j^{old} K_{ij} + 2 \alpha_j K_{ij} + \sum_{k \neq i,j} \alpha_k y^{(j)} y^{(k)} K_{j,k} - \sum_{k \neq i,j} \alpha_k y^{(i)} y^{(k)} K_{i,k}$$

$$= y^{(j)} (y^{(i)} - y^{(j)}) + \alpha_j^{old} K_{ii} - \alpha_j K_{ii} - \alpha_j K_{jj} - \alpha_j^{old} K_{ij} + 2 \alpha_j K_{ij} + y^{(j)} \sum_{k=1}^m \alpha_k y^{(k)} K_{i,k}$$

$$- y^{(i)} \sum_{k=1}^m \alpha_k y^{(k)} K_{j,k} - \alpha_j^{old} K_{ij} + \alpha_j^{old} K_{jj} \quad (\text{Notice } 1 - y^{(i)} y^{(j)} = y^{(j)} (y^{(j)} - y^{(i)}))$$

$$= -\alpha_j^{old} (\sum K_{ij} - K_{ii} - K_{jj}) + \alpha_j (\sum K_{ij} - K_{ii} - K_{jj}) + y^{(j)} (E_i - E_j)$$

$$= (\alpha_j - \alpha_j^{old}) \eta + y^{(j)} (E_i - E_j) \stackrel{\text{set}}{=} 0$$

$$\alpha_j \eta = \alpha_j^{old} \eta - y^{(j)} (E_i - E_j)$$

$$\alpha_j := \alpha_j^{old} - \frac{y^{(j)} (E_i - E_j)}{\eta}$$

Now that we have found the undipped value for α_j , let's clip it to live between L and H .

$$\alpha_j := \begin{cases} H & \text{if } \alpha_j > H \\ \alpha_j & \text{if } L \leq \alpha_j \leq H \\ L & \text{if } \alpha_j < L \end{cases}$$

Now that we have computed our new α_j , we should update α_i using our previous representation:

$$\alpha_i := \alpha_i^{\text{old}} + \eta y^{(i)} y^{(j)} (\alpha_j^{\text{old}} - \alpha_j)$$

The full SMO algorithm can handle the rare case where $\eta=0$, however, for our implementation, we will treat this as a case where we cannot make any progress with α_i and α_j .

After updating α_i and α_j , we choose the threshold b such that the KKT conditions are satisfied for the i th and j th examples. Let's derive the threshold update:

When an example isn't on the bounds ($0 < \alpha_i < C$), it sits on the margin, meaning:

$$f^{\text{new}}(x^{(i)}) = y^{(i)}$$

$$f^{\text{new}}(x^{(i)}) = \sum_{k=1}^m \alpha_k y^{(k)} K(x^{(i)}, x^{(k)}) + b_1^{\text{new}}$$

$$= \alpha_i y^{(i)} K(x^{(i)}, x^{(i)}) + \alpha_j y^{(j)} K(x^{(i)}, x^{(j)}) + \sum_{k \neq i, j} \alpha_k y^{(k)} K(x^{(i)}, x^{(k)}) + b_1^{\text{new}} = y^{(i)}$$

$$\alpha_i^{\text{new}} y^{(i)} K_{ii} + \alpha_j^{\text{new}} y^{(j)} K_{ji} + E_i - \alpha_i^{\text{old}} y^{(i)} K_{ii} - \alpha_j^{\text{old}} y^{(j)} K_{ji} - b^{\text{old}} + b_1^{\text{new}} = 0$$

$$b_1^{\text{new}} = b^{\text{old}} - E_i - \alpha_i^{\text{new}} y^{(i)} K_{ii} - \alpha_j^{\text{new}} y^{(j)} K_{ji} + \alpha_i^{\text{old}} y^{(i)} K_{ii} + \alpha_j^{\text{old}} y^{(j)} K_{ji}$$

$$b_1^{\text{new}} = b^{\text{old}} - E_i - y^{(i)} (\alpha_i - \alpha_i^{\text{old}}) K_{ii} - y^{(j)} (\alpha_j - \alpha_j^{\text{old}}) K_{ji} \quad (\alpha_i = \alpha_i^{\text{new}})$$

We can repeat this process for α_j when $0 < \alpha_j < C$

$$f^{\text{new}}(x^{(j)}) = y^{(j)}$$

$$f^{\text{new}}(x^{(j)}) = \sum_{k=1}^m \alpha_k y^{(k)} K(x^{(j)}, x^{(k)}) + b_2^{\text{new}}$$

$$= \alpha_i^{\text{new}} y^{(i)} K_{ij} + \alpha_j^{\text{new}} y^{(j)} K_{jj} + \sum_{k \neq i, j} \alpha_k^{\text{old}} y^{(k)} K_{kj} + b_2^{\text{new}} = y^{(j)}$$

$$\alpha_i^{\text{new}} y^{(i)} K_{ij} + \alpha_j^{\text{new}} y^{(j)} K_{jj} + E_j - \alpha_i^{\text{old}} y^{(i)} K_{ij} - \alpha_j^{\text{old}} y^{(j)} K_{jj} - b^{\text{old}} + b_2^{\text{new}} = 0$$

$$b_2^{\text{new}} = b^{\text{old}} - E_j - y^{(i)} (\alpha_i - \alpha_i^{\text{old}}) K_{ij} - y^{(j)} (\alpha_j - \alpha_j^{\text{old}}) K_{jj}$$

Now that we have computed b_1 and b_2 , we can update our b parameter using the following set of rules:

$$b := \begin{cases} b_1 & \text{if } 0 < d_i < C \\ b_2 & \text{if } 0 < d_j < C \\ (b_1 + b_2)/2 & \text{otherwise} \end{cases}$$

Note that if $0 < d_i < C$ and $0 < d_j < C$, both b_1 and b_2 are valid as they are equal. If both d_i and d_j are at the bounds (i.e. $d_i = 0$ or $d_i = C$ and $d_j = 0$ or $d_j = C$), then all the thresholds between b_1 and b_2 are valid, so naturally and for simplicity, we average them.

This gives us a clean way to implement the SVM and solve the constrained optimization problems with the (simplified) SMO algorithm:

- 1) choose d_i (in a "for" loop)
- 2) check whether d_i violates the KKT conditions
- 3) choose $d_j \neq d_i$ randomly
- 4) update d_j and d_i
- 5) update b
- 6) repeat until convergence